Blood Bank Donor & Inventory Management System

Final Exam Project Report — Web Technologies and API Services

API data, machine-learning models deployed in Django, and a predictive dashboard

Master's in Big Data Analytics

GROUP 2

November 2026

GROUP MEMBERS

ID	Name
20251MBI012	Izabayo Isabelle
20251MBI050	Hirwa Fabrice
20251MBI014	Masengesho Donatien

1. Introduction

This project is a database-driven web application built with Django to support the day-to-day operations of a blood bank network: registering donors, running donation campaigns, recording and lab-testing collected blood units, and matching that stock against blood requests raised by hospitals. At the midterm stage the system answered the question *what blood do we have right now, and which requests are still open?*

For the final exam the same system was extended to answer the forward-looking questions a blood bank actually plans around: *how much blood will hospitals ask for next week, which donors are likely to come back, and which donor at the screening desk is likely to be deferred?* To do this the project gained a JSON **API**, a synthetic two-year history to learn from, three **machine-learning models** trained on data read through that API, and a deployment layer inside Django that scores every new donation, request and screening as it arrives and shows the results on an upgraded AdminLTE dashboard.

The system is now organised as four Django apps: **core** (institutions and staff), **donors** (donor registry, campaigns and the donation-to-blood-unit pipeline), **inventory** (blood requests, issuance and stock thresholds) and the new **analytics** app (API, model training, stored predictions and the analytics pages).

1.1 Objectives

- Keep the midterm foundations: a realistic multi-entity domain model, full CRUD through validated forms, the domain's business rules enforced at model level, AdminLTE integrated through template inheritance, and authenticated access.
- **Read data from an API.** Expose the data as a paginated, authenticated JSON API built with Django, and make the training pipeline read all of its data through that API.
- **Train models.** Generate realistic synthetic data that obeys every validation rule, then train and honestly evaluate three models against simple baselines.
- **Deploy the models in Django.** Store each trained model as a versioned file, load the active version inside the application and serve predictions from it.
- **Predict on new incoming data.** Score new donations, requests and screenings automatically as they arrive, and display the predictions on an improved dashboard and on dedicated analytics pages.

1.2 Final-exam requirements and where they are met

Requirement	How the system meets it	Where to see it
1. Read data from an API	The project has its own REST-style JSON API (10 endpoints). `train_models` reads donors, donations and blood requests through it, either over HTTP from a running server or in-process.	Section 4.1; Figures 20–22
2. Train the model on original or synthetic data	`generate_synthetic_data` builds a two-year history (3,000 donors, ~7,000 donations, ~3,100 requests). Three models are trained with a time-based train/test split and compared with baselines.	Sections 4.2–4.4; Figures 18–19
3. Deploy the model in Django	Models are saved as versioned JSON files and registered in the `TrainedModel` table; `analytics/services.py` loads the active version. Staff can retrain or roll back from the UI.	Section 4.5; Figure 18
4. Predict on new incoming data and display it on an improved dashboard	Django signals refresh predictions whenever data changes; the ingest API returns predictions immediately. The dashboard gained forecast, shortage, donor-targeting, screening and model-quality panels plus four Analytics & Predictions pages.	Sections 4.5–4.6; Figures 4, 13–17, 23–24

Table 1. The four final-exam requirements and the parts of the system that satisfy them.

2. System Design

2.1 Application structure

The project is split into four Django apps, each owning one slice of the domain:

App	Responsibility	Key models
core	Institutions (hospitals / blood banks) and which staff user belongs to which institution; also hosts the shared dashboard and login views.	Institution, InstitutionMembership
donors	The donor registry, donation campaigns, individual donation records and the blood units a collected donation produces.	Donor, DonationCampaign, Donation, BloodUnit
inventory	Blood requests raised by hospitals, the units issued against them, and per-blood-group minimum stock thresholds.	BloodRequest, IssuedUnit, StockThreshold
analytics *(new)*	The JSON API, synthetic-data generation, model training, the deployed models and their stored predictions, and the Analytics & Predictions pages.	TrainedModel, DemandForecast, DonorPrediction, ScreeningCheck

Table 2. Four Django apps, each with a single clear responsibility. The analytics app is new for the final exam.

2.2 Database design and relationships

The data model now has fourteen entities: the thirteen application models above plus Django's built-in `auth.User`, which several models reference for accountability (who recorded a donation, handled a request, issued a unit or ran a screening). Figure 1 shows the updated entity-relationship diagram; the four analytics entities are drawn in purple and marked NEW.

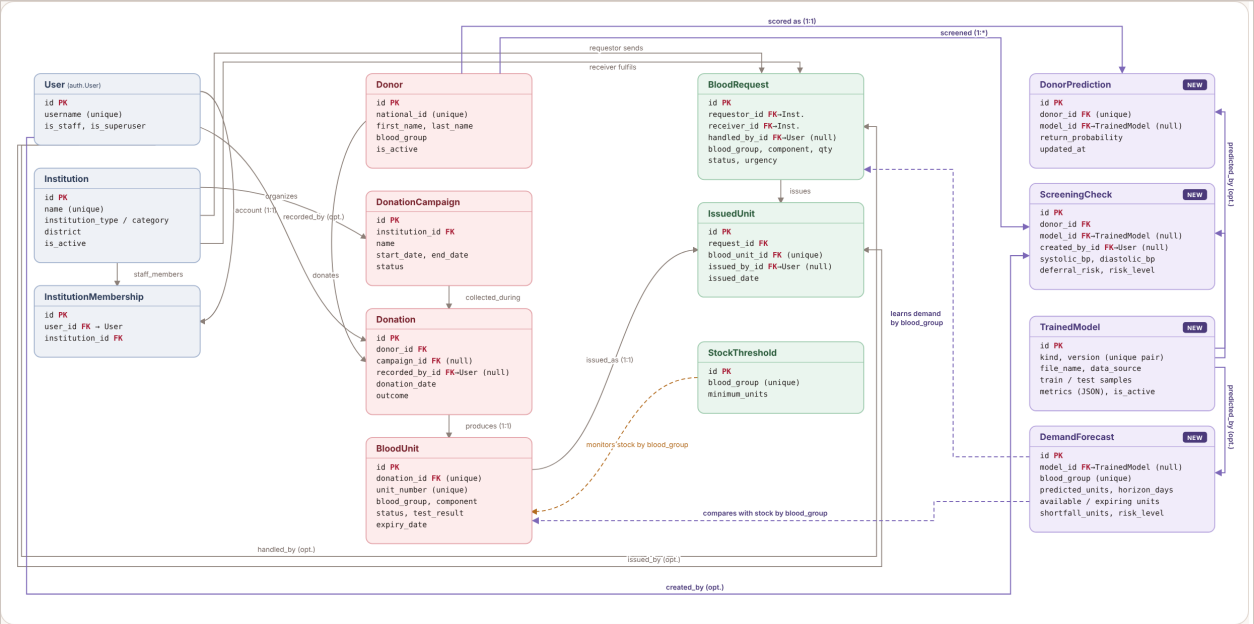


Figure 1. Entity-relationship diagram — core (blue), donors (red), inventory (green) and the new analytics app (purple, marked NEW), plus Django's auth.User. The image is high-resolution: zoom in to read field names, or open docs/Final_exam/er_diagram.png.

The analytics entities follow two design decisions. First, every prediction row points at the **model version** that produced it (`model_id`, `SET_NULL`), so a result can always be traced back to the exact model and its test metrics. Second, predictions are stored rather than computed on every page view: `DonorPrediction` is one row per donor (one-to-one) and `DemandForecast` one row per blood group, refreshed when new data arrives, which keeps the dashboard fast with thousands of donors. `ScreeningCheck` is an append-only history, because each screening is an event worth keeping.

Relationship	Cardinality	Meaning
User → InstitutionMembership	1 : 1	A staff account has at most one institution membership (OneToOneField).
Institution → InstitutionMembership	1 : *	One institution has many staff members.
Institution → DonationCampaign	1 : *	An institution organises many campaigns over time.
Donor → Donation	1 : *	A donor can make many donations, subject to the 56-day eligibility rule enforced in Donor.is_eligible_on().

Relationship	Cardinality	Meaning
DonationCampaign → Donation	1 : * (optional)	A campaign can group many donations; a donation may also stand alone (null=True).
User → Donation	1 : * (optional)	The staff member who recorded the donation.
Donation → BloodUnit	1 : 1	A collected donation produces exactly one trackable unit; a deferred donation produces none.
Institution → BloodRequest (requestor)	1 : *	The institution sending the request — usually a hospital.
Institution → BloodRequest (receiver)	1 : *	The institution asked to fulfil it — usually the blood bank. A second, independent FK to Institution.
User → BloodRequest	1 : * (optional)	The staff member who handled the request.
BloodRequest → IssuedUnit	1 : *	A request can be fulfilled by several individual issuances.
BloodUnit → IssuedUnit	1 : 1	A specific unit can be issued once (OneToOneField).
User → IssuedUnit	1 : * (optional)	The staff member who released the unit.
StockThreshold → BloodUnit	logical only	Not a foreign key — compared at query time on blood_group to drive the low-stock alerts.
Donor → DonorPrediction *(new)*	1 : 1	Each donor who has donated has one current return-likelihood prediction, refreshed after every donation.
Donor → ScreeningCheck *(new)*	1 : *	A donor can be screened many times; each screening stores its predicted deferral risk.
User → ScreeningCheck *(new)*	1 : * (optional)	The staff member who ran the screening; empty when it arrived through the API.
TrainedModel → DemandForecast *(new)*	1 : * (optional)	The model version that produced the forecast.
TrainedModel → DonorPrediction *(new)*	1 : * (optional)	The model version that scored the donor.
TrainedModel → ScreeningCheck *(new)*	1 : * (optional)	The model version that predicted the deferral risk.
DemandForecast → BloodRequest *(new)*	logical only	The forecast for a blood group is computed from that group's recent request history.
DemandForecast → BloodUnit *(new)*	logical only	The forecast is compared with available, unexpired stock of the same group to grade shortage risk.

Table 3. All twenty-two relationships: nineteen foreign-key / one-to-one relationships and three logical links. Eight are new.

2.3 Machine-learning architecture

Figure 2 shows how data flows through the new parts of the system. New data enters through the web forms or the ingest API and is stored in the database. Training reads that data back through the read API, fits the models and saves them as versioned files. The prediction service loads the active versions and is triggered by Django signals whenever data changes, so the dashboard always reflects the latest records.

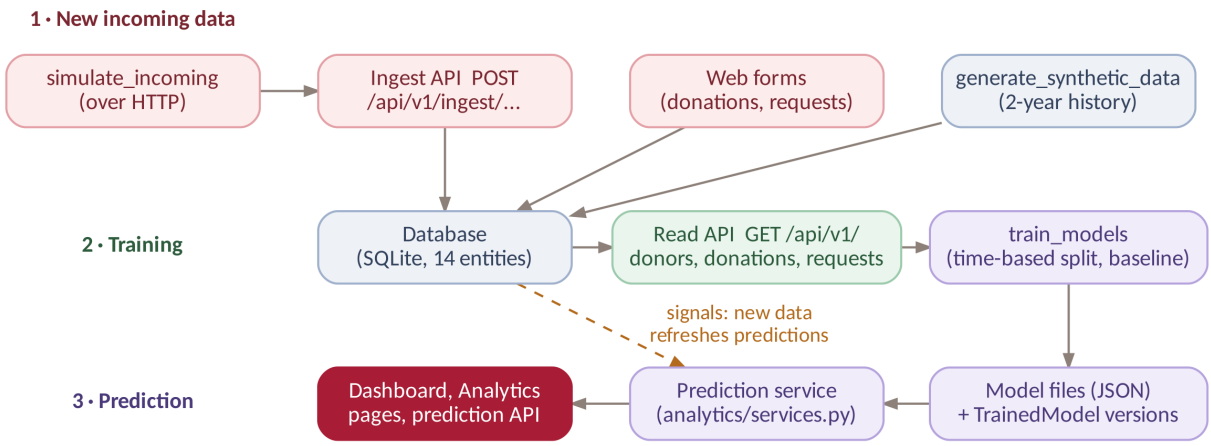


Figure 2. Data flow: incoming data → database → read API → training → versioned model files → prediction service → dashboard and API.

3. Implementation

3.1 URL routing and views

Each app owns its own `urls.py`, included from the project-level `blood_bank_donor/urls.py`. The system now has **42 routes** outside the Django admin: 23 from the midterm system, 9 analytics pages and staff actions, and 10 API endpoints. Many take a dynamic `<int:pk>` parameter that resolves to a record through `get_object_or_404`.

URL	View	Purpose
/	DashboardView	KPIs, alerts, stock and forecast charts, donor targeting, live screenings, model quality.
/accounts/login/, /accounts/logout/	SystemLoginView / LogoutView	AdminLTE-styled login; logout by POST.
/institutions/, /add/, /<pk>/edit/	Institution list / create / update	Manage partner institutions.
/donors/donors/, /<pk>/, /add/, /<pk>/edit/	Donor list / detail / create / update	Donor registry and profile.
/donors/campaigns/ (+ add, edit)	Campaign list / create / update	Manage donation campaigns.

URL	View	Purpose
/donors/donations/ (+ add, delete, unit clear)	Donation views, clear_blood_unit	Record donations; lab-clear units (POST).
/inventory/inventory/requests/ (+ add, edit)	BloodRequest list / create / update	Manage hospital blood requests.
/inventory/.../requests/<pk>/issue/, /process/	IssueUnitsView, process_issuance	Match and issue stock against a request (POST, login required).
/screening/ *(new)*	ScreeningView	Pre-donation check that returns a deferral-risk prediction.
/analytics/forecasts/ *(new)*	DemandForecastView	7-day red-cell demand vs stock per blood group.
/analytics/donor-predictions/ *(new)*	DonorPredictionListView	Return likelihood for every donor, with filters.
/analytics/screenings/ *(new)*	ScreeningCheckListView	History of all screenings and their risk.
/analytics/models/, /<pk>/ *(new)*	TrainedModelListView / DetailView	Model versions, metrics and learned feature weights.
/analytics/models/retrain/, /<pk>/activate/, /analytics/refresh/ *(new)*	retrain_models, activate_model, refresh_predictions	Staff-only POST actions: retrain, roll back, recompute.
/api/v1/... *(new)*	analytics/api.py	10 JSON endpoints — see Table 5.

Table 4. Representative routes. Rows marked (new) were added for the final exam.

Known quirk (unchanged from the midterm): `donors.urls` and `inventory.urls` are included under a prefix that repeats their own first segment, so the live paths are `/donors/donors/...` and `/inventory/inventory/...`. Everything resolves correctly; templates use route names, so shortening the paths later only requires changing the two `urls.py` files.

3.2 Templates, template inheritance and AdminLTE integration

All authenticated pages extend `templates/adminlte/base.html`, which carries the AdminLTE 3 shell, navbar, sidebar and static assets. For the final exam the sidebar gained an expandable **Analytics & Predictions** menu (Demand Forecasts, Donor Predictions, New Screening Check, Screening Checks, Trained Models) that stays open and highlights the current page. The new templates live in `templates/adminlte/analytics/` and reuse the same AdminLTE components (small boxes, info boxes, cards, callouts, progress bars) as the existing pages.

The dashboard is rendered once on the server from the same JSON that `/api/v1/predictions/dashboard/` returns (embedded with Django's `json_script` filter), then refreshes itself from that endpoint every 30 seconds with Chart.js redrawing the charts, so new predictions appear without reloading the page.

3.3 CRUD operations and validation

Every primary entity (Donor, DonationCampaign, Donation, Institution, BloodRequest) has list, create and update views built on ModelForm-backed generic views, with a detail view for the donor profile. Validation stays at model level via `clean()` and `full_clean()`, so it applies equally to the forms, the admin, the ingest API and the synthetic-data generator:

- Donor: age 18–65 and weight ≥ 50 kg when a donor is registered; existing donors can still be edited or deactivated after turning 66, while `is_eligible_on()` decides whether they may donate.
- Donation: haemoglobin < 12.5 g/dL forces a deferral; a collected donation must respect the 56-day interval (excluding the record being edited) and the campaign's dates.
- BloodUnit: expiry is derived from the component's shelf life (35 days whole blood, 42 red cells, 365 plasma, 5 platelets); a reactive unit can never be available.
- BloodRequest / IssuedUnit: a unit can only be issued if group and component match, it is unexpired and lab-cleared, and the request is not already complete.

State-changing actions that are not forms (delete a donation, clear a unit, remove an issuance, retrain, activate a model version, refresh predictions) accept **POST only** and carry a CSRF token, so a link or image on another site cannot trigger them.

3.4 Django admin, authentication and API security

All thirteen application models are registered in the Django admin; the four analytics models appear under *Analytics & predictions*, with trained-model metrics shown read-only. Every operational page requires login (`LoginRequiredMixin` / `@login_required`), and the retrain, activate and refresh actions additionally require a staff account.

The API accepts either a logged-in session or an `X-API-Key` header, compared in constant time; ingest (POST) endpoints always require the key, and unauthenticated calls receive HTTP 401 (Figure 24). The secret key, debug flag, allowed hosts and API key are read from environment variables, with safe development defaults.

4. API and Machine Learning

4.1 The REST API (requirement 1)

The API is written with plain Django views (`analytics/api.py`), so no extra framework is needed. Read endpoints are paginated (500 records per page by default, up to 2,000) and return a `next` link; donations and requests can be filtered by date. Database rows are converted to JSON by shared serializers, and the training code builds its features from exactly those JSON records, so training and live prediction see identical data.

Method	Endpoint	Purpose
GET	/api/v1/donors/	Donors (paged: ?page=, ?page_size=)
GET	/api/v1/donations/	Donations (?since=YYYY-MM-DD&until=)
GET	/api/v1/blood-requests/	Hospital requests (?since=&until=)
GET	/api/v1/blood-units/	Blood units (?status=AVAILABLE)
POST	/api/v1/ingest/donors/	Register a new donor
POST	/api/v1/ingest/donations/	Record a donation; returns the donor's updated return likelihood
POST	/api/v1/ingest/blood-requests/	Record a request; returns the updated 7-day forecast for that group
POST	/api/v1/ingest/screenings/	Pre-donation check; returns deferral risk and its main factors
GET	/api/v1/predictions/dashboard/	Everything the dashboard shows (KPIs, forecasts, donor lists, screenings, trends, models)
GET	/api/v1/predictions/donors/<id>/	One donor's prediction and eligibility

Table 5. API endpoints. Read endpoints feed the training pipeline; ingest endpoints receive new incoming data.

Training reads the data through these endpoints in one of two ways: over HTTP from a running server (`--api-url http://127.0.0.1:8000`, which follows the pagination links), or by calling the same API views in-process when no server is running. Both clients return identical records.

```
# over HTTP from a running server (follows the API's pagination links)
$ python manage.py train_models --api-url http://127.0.0.1:8000

# in-process, calling the same API views without a server
$ python manage.py train_models
```

4.2 Synthetic training data (requirement 2)

No real patient or donor data was available, so `generate_synthetic_data` creates a realistic two-year history (10 November 2024 to 10 November 2026). Every row passes the same model validation as real data (a sample of 800 rows was re-checked with `full_clean()`), and the generator deliberately contains patterns for the models to discover:

- **Donors (3,000):** a hidden "loyalty" trait controls how soon and how often each donor returns; older donors are slightly more loyal, and a deferral makes a donor less likely to come back.
- **Visits (~7,000):** haemoglobin and blood pressure are drawn around donor-specific baselines; Hb below 12.5 g/dL or blood pressure above 160/100 leads to deferral, plus about 2% deferrals for other reasons.
- **Campaigns (19):** drives in different districts every 4–7 weeks attract donors who are due.

- **Hospital demand (~3,100 requests):** a weekly cycle (busiest on Mondays, quietest on Sundays), a seasonal cycle with an end-of-year peak, and a slow upward trend; demand runs slightly ahead of supply, as it does in practice.
- **Stock:** units are lab-tested (about 2.5% reactive), issued first-expiry-first-out, and expire if unused.

4.3 Models and training

All three models are deliberately simple, interpretable linear models, implemented with numpy in `analytics/ml/algorithms.py` (ridge regression solved in closed form; logistic regression trained with Newton's method). Every model is evaluated on the **most recent** part of the data, which it never saw during training — a random split would let the model peek at the future — and compared with a simple baseline.

Model	Predicts	Features (inputs)	Algorithm and evaluation
Blood demand forecast	Red-cell units (whole blood + red cells) hospitals will request per blood group in the next 7 days	Units requested in the last 7, 14 and 28 days, week-over-week change, season, blood group (14 features)	Ridge regression; test on the last 20% of days; baseline: "next week = last week"
Donor return likelihood	Probability that a donor gives blood again within 180 days	Days since last donation, number of donations, volume, time since first visit, deferrals, donation rate, age, sex, campaign share, eligible now (10)	Logistic regression on monthly snapshots; test on the last 25% of snapshot dates; baseline: recency alone
Donation deferral risk	Probability that a screening visit ends in deferral, before the Hb test	Age, sex, weight, systolic and diastolic BP, BP above 160/100, first-time donor, days since last donation, earlier deferrals and donations, last Hb result (11)	Class-balanced logistic regression; test on the last 20% of visits

Table 6. The three models. The current visit's haemoglobin is deliberately excluded from the deferral model because it decides deferral by rule.

4.4 Results

Model (active version)	Train / test samples	Result on the held-out period	Baseline
Blood demand forecast (v2)	4,456 / 1,112	Mean absolute error 2.39 units per group per week (RMSE 3.54; average demand 6.6 units)	3.35 units — the model is 28.6% more accurate
Donor return likelihood (v1)	14,173 / 8,680	ROC AUC 0.708; accuracy 0.691; precision 0.625; recall 0.345	AUC 0.685 using recency alone
Donation deferral risk (v1)	5,647 / 1,420	ROC AUC 0.751; recall 0.735; precision 0.305; accuracy 0.706	15.1% of visits are deferrals

Table 7. Test results of the deployed models, read from the Trained Models page (Figure 18).

How to read these numbers. The demand model clearly beats the naive baseline because it captures the weekly and seasonal pattern. The donor-return model improves on ranking donors by recency alone, and its precision of 0.63 means that most donors it flags as likely to return actually do — which is what a call list needs. The deferral model is tuned for **recall**: it catches about three in four visits that end in deferral, at the cost of false alarms (precision 0.31). For a screening desk that is the right trade-off, because a flag only asks staff to check haemoglobin and blood pressure carefully; the clinical test still makes the decision. Figure 19 shows what the deferral model learned: blood pressure above the 160/100 limit raises the risk most, while male donors (who rarely have low haemoglobin) and a longer gap since the last donation lower it.

Limitation. Because the models are trained on synthetic data, these figures show that the pipeline learns the patterns placed in the data, not how accurate it would be on a real blood bank's records. The same commands retrain the models on real data once it is available, and the time-based evaluation will report their real accuracy.

4.5 Deployment and predictions on new incoming data (requirements 3 and 4)

- **Versioned model files.** Each training run writes a JSON file to `ml_models/` (weights, feature names and test metrics) and a `TrainedModel` row; the newest becomes the active version. JSON is used instead of pickle so that loading a model file can never execute code.
- **Serving.** `analytics/services.py` loads the active version once, caches it, and computes forecasts, donor predictions and screening risks with the same feature code used in training.
- **Automatic refresh.** Django signals on Donation, BloodRequest, BloodUnit and IssuedUnit schedule a refresh after the database transaction commits, so a new donation entered in a form, the admin or the API immediately updates that donor's prediction and the demand forecasts. A failure in prediction can never block data entry.
- **Ingest API responses.** Each POST returns its prediction straight away — a new screening returns its deferral risk and top factors, a new request returns the updated forecast for its blood group (Figures 23–24).
- **Operations.** Staff can retrain from the Trained Models page, roll back to an earlier version, or recompute all predictions; `refresh_predictions` can also run nightly.

4.6 The improved dashboard and the analytics pages

The dashboard keeps the midterm KPIs and low-stock alerts and adds: forecast-based shortage alerts; a second KPI row (forecast demand, groups at high risk, donors expected back, 30-day deferral rate); a stock-versus-forecast chart and shortage table with *days of cover*; 90-day trends of collections, demand and deferrals; a list of **donors to invite now** (eligible donors ranked by return likelihood, boosted for blood groups that are running short); regular donors at risk of lapsing; a live feed of screenings with their risk factors; and cards showing each deployed model's quality. Four Analytics & Predictions pages (Section 5.2) give the full detail behind each panel.

5. Screenshots

All screenshots were taken from the running system with the synthetic data described in Section 4.2.

5.1 Core system screens

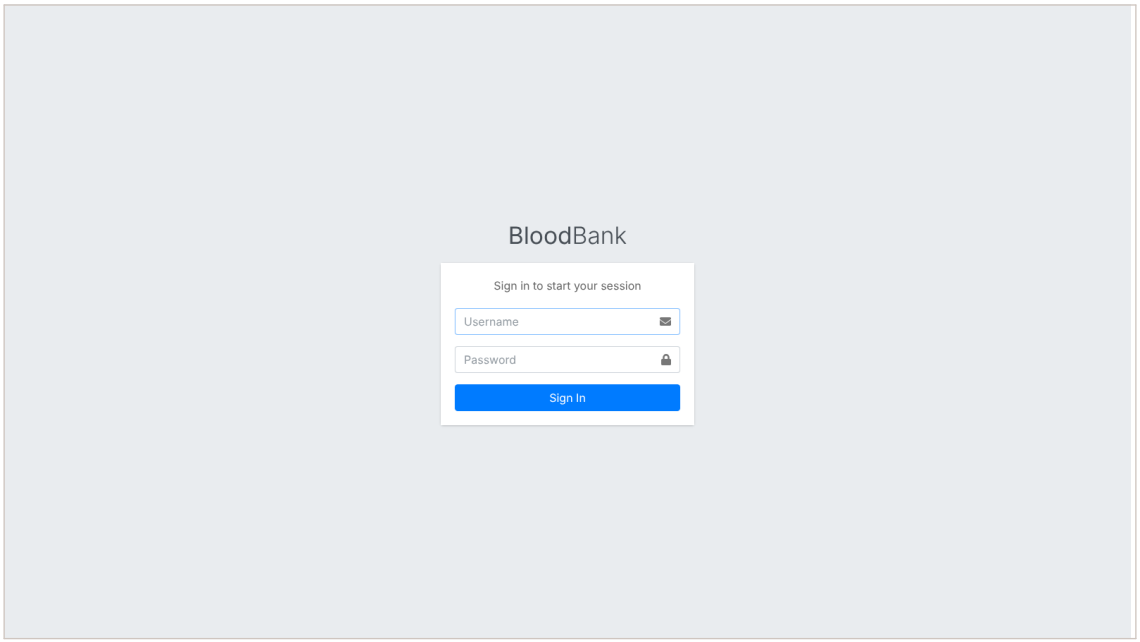


Figure 3. AdminLTE-styled login page (SystemLoginView).

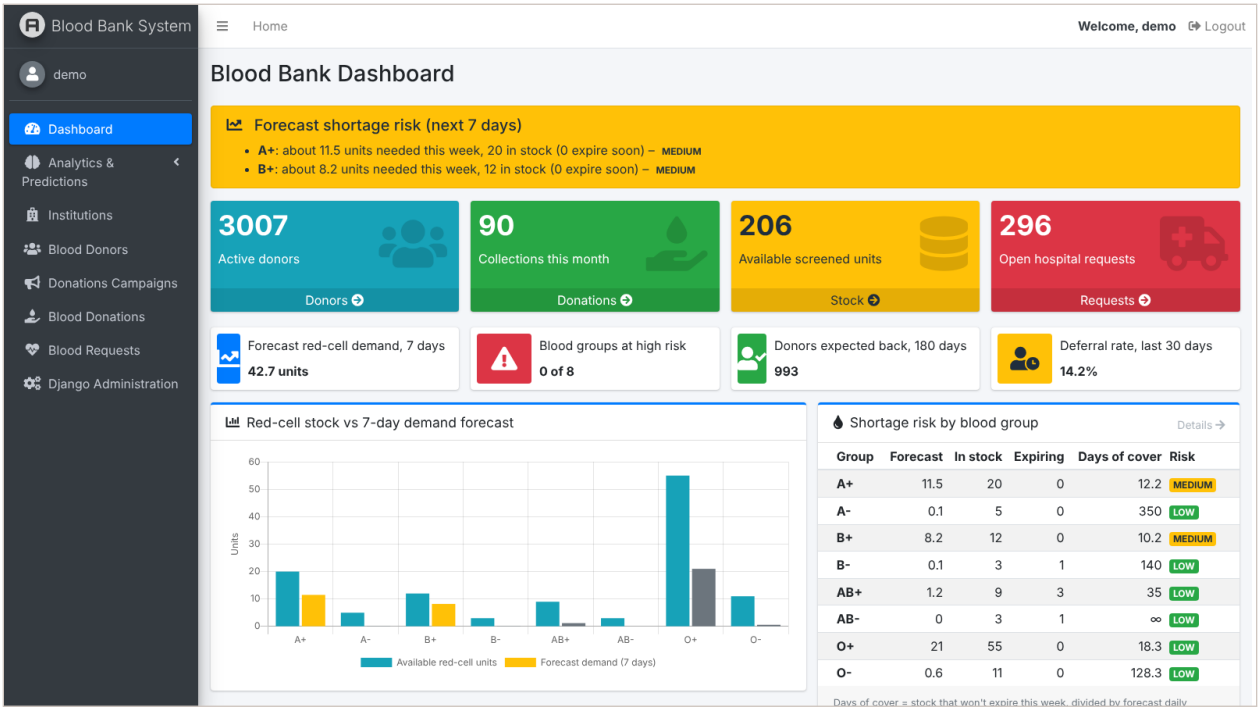


Figure 4. Dashboard — alerts, operational and prediction KPIs, stock vs forecast chart and shortage table.

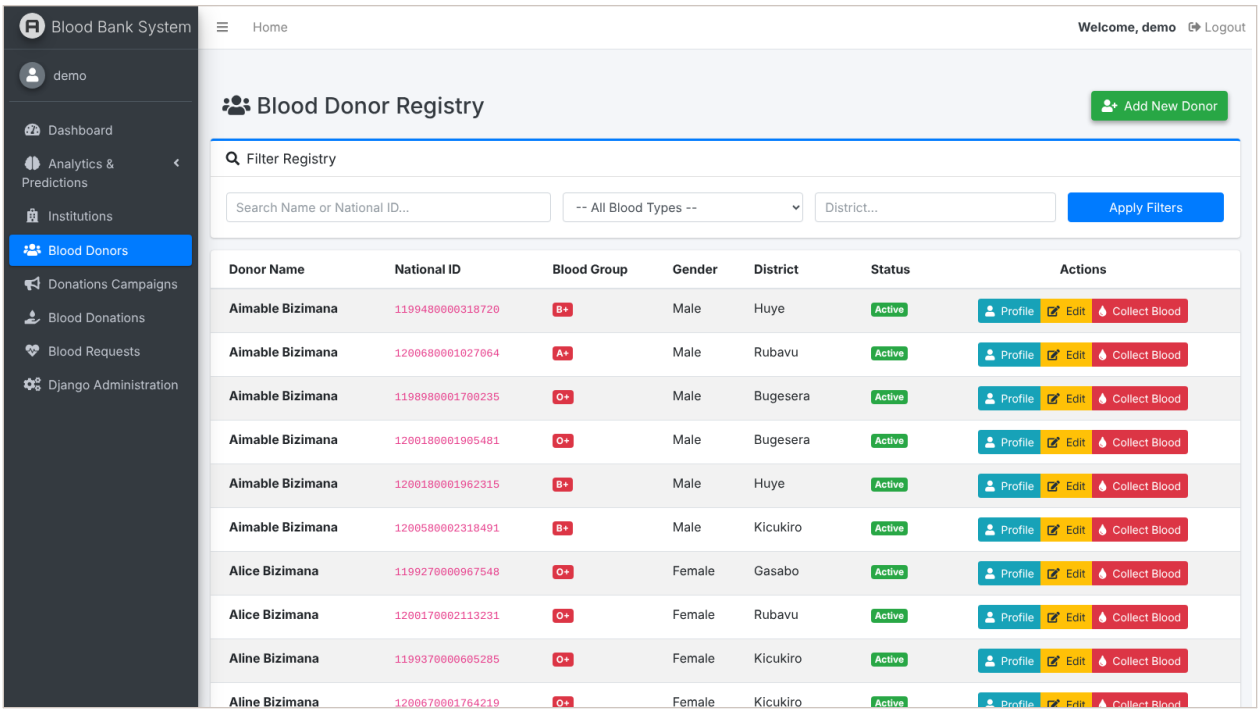


Figure 5. Donor registry (paginated, filterable, database-backed table).

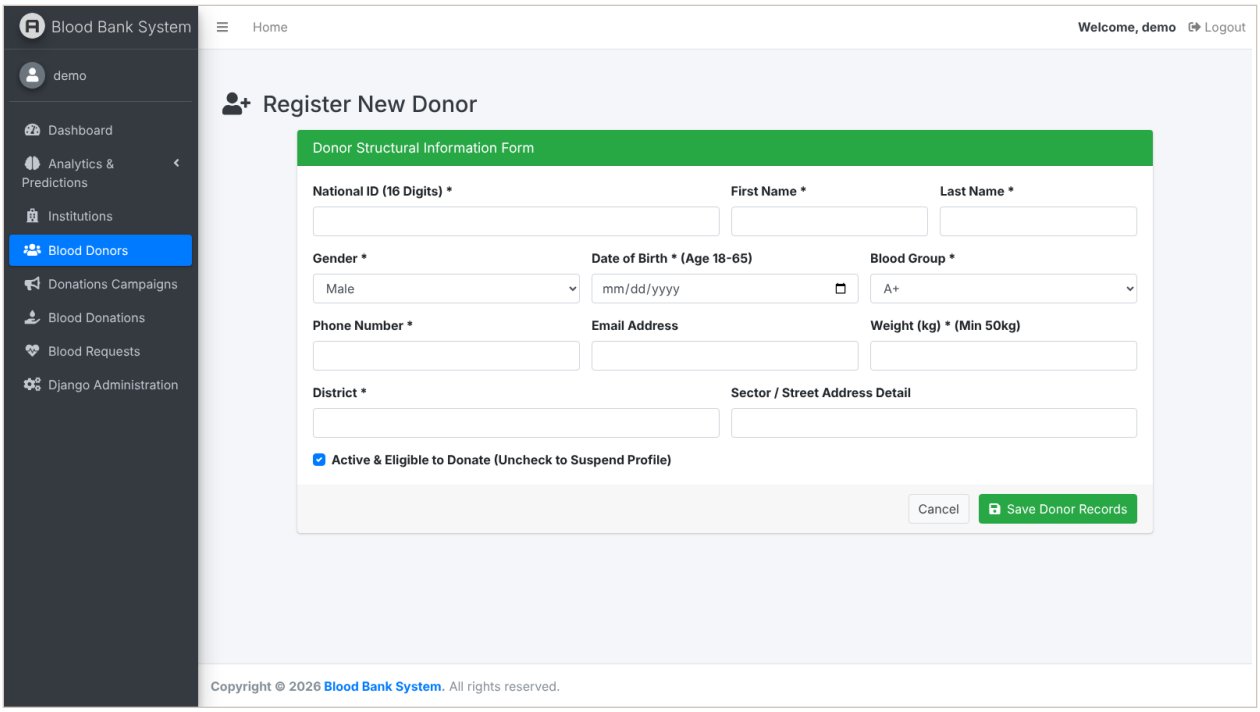


Figure 6. Add Donor form (ModelForm with model-level validation).

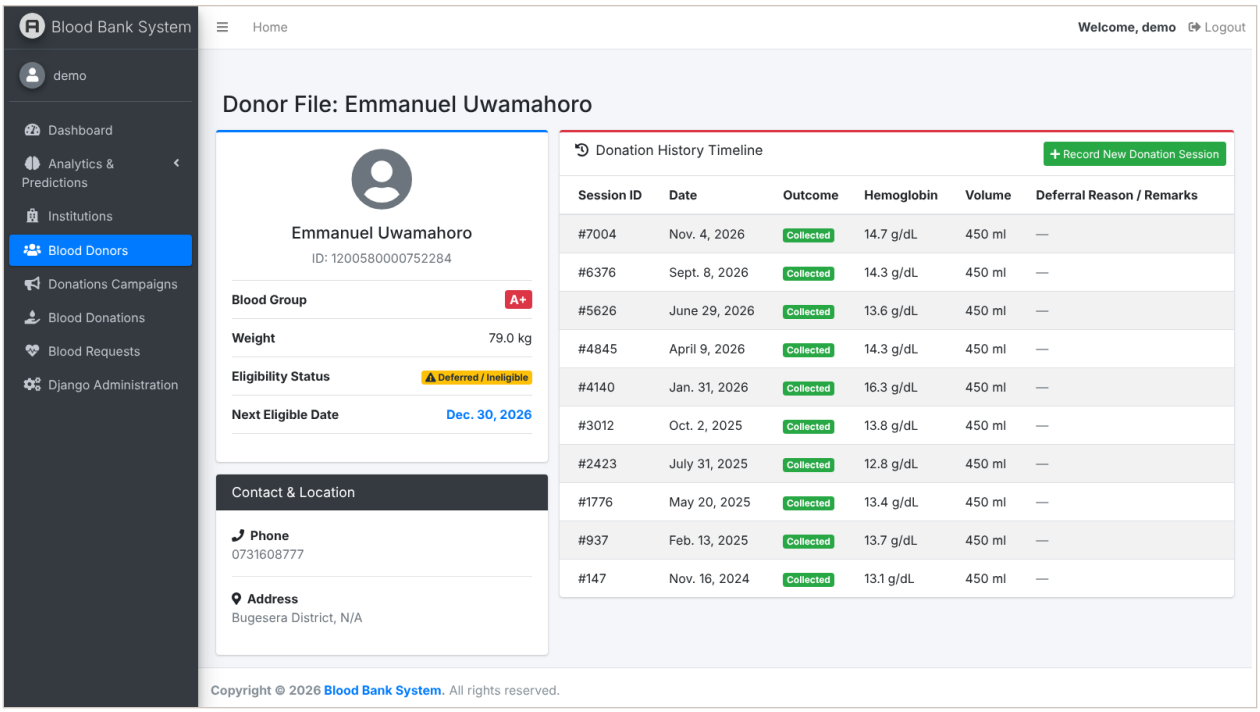


Figure 7. Donor profile with eligibility status and donation history.

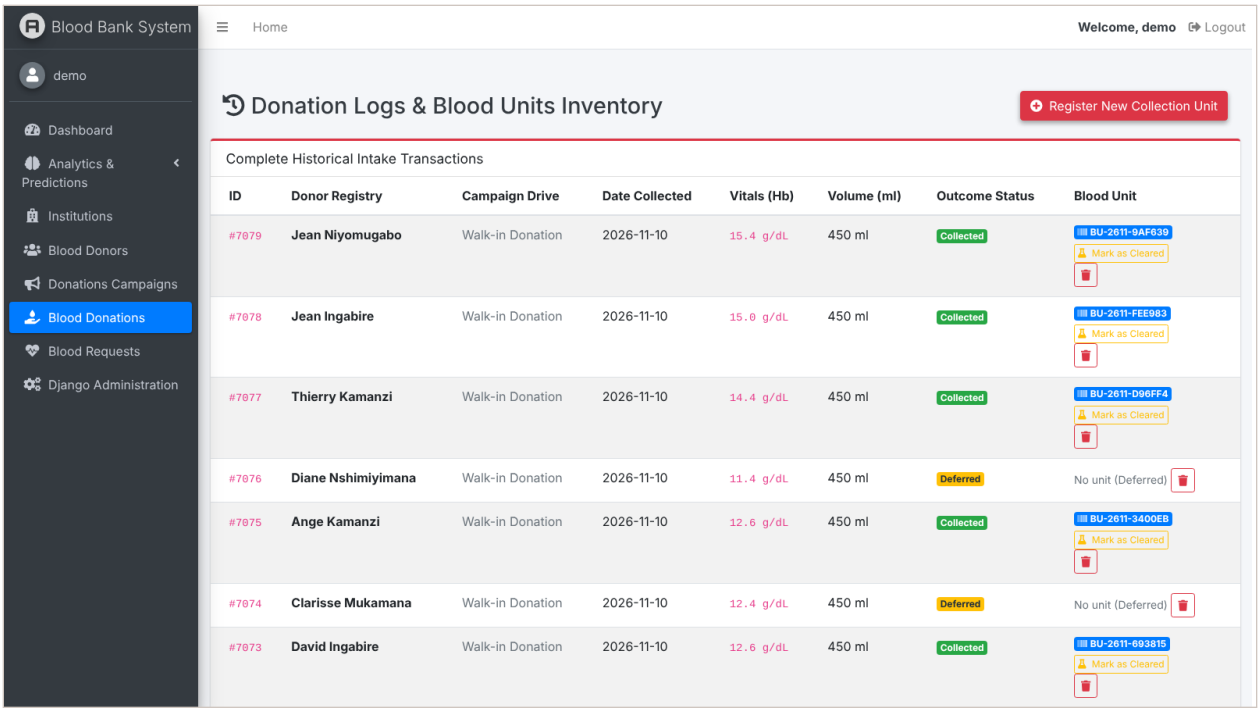


Figure 8. Donation log with blood-unit status and POST-only clear / delete actions.

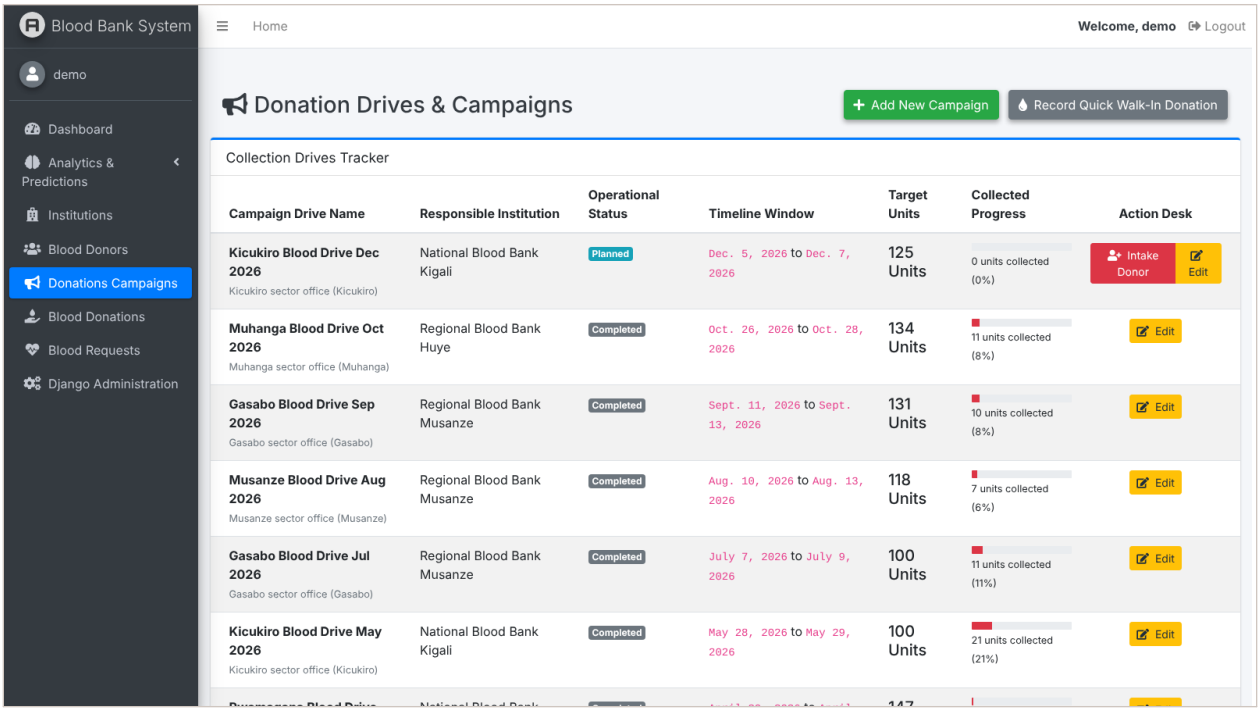


Figure 9. Donation campaigns with progress towards target units.

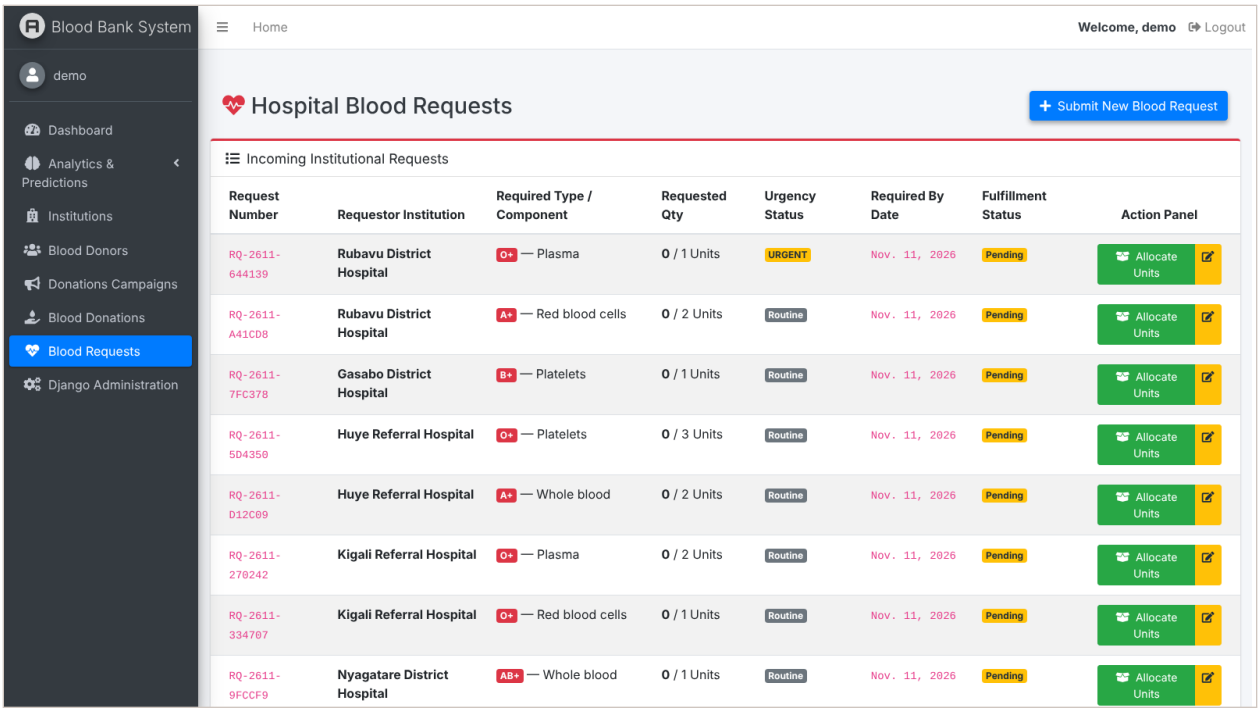


Figure 10. Blood requests across institutions.

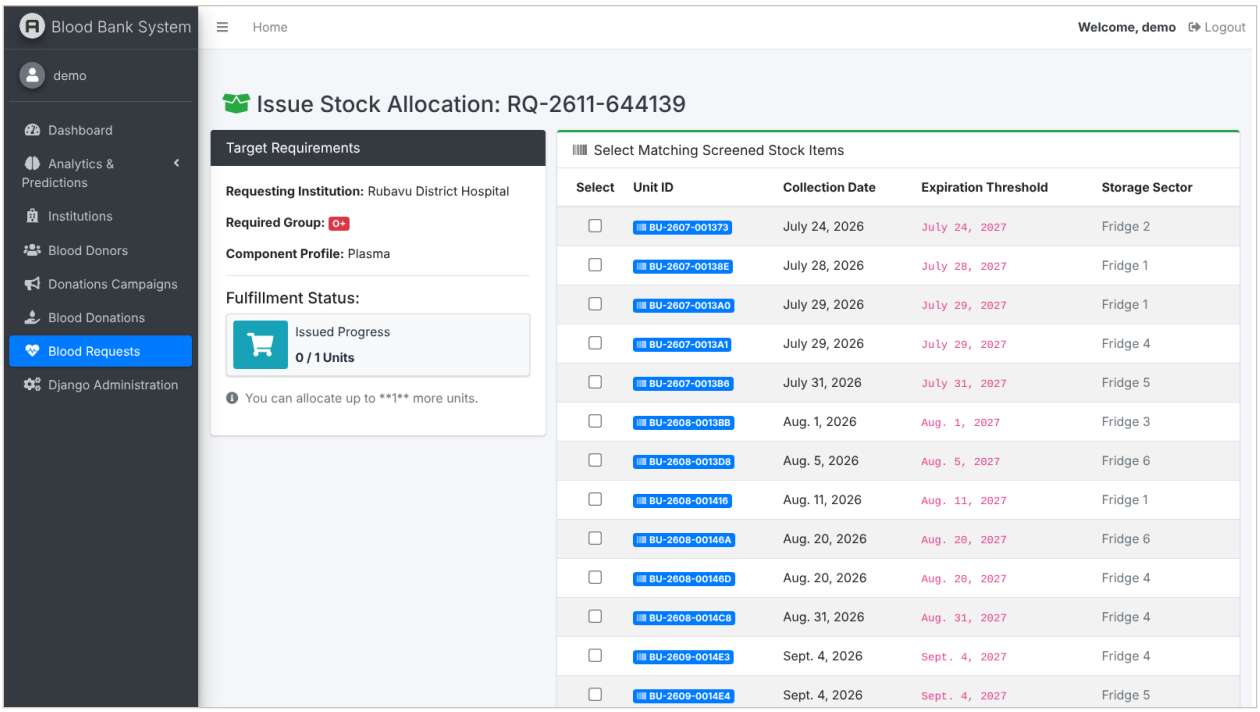


Figure 11. Issue Units — matching available screened stock against a request.

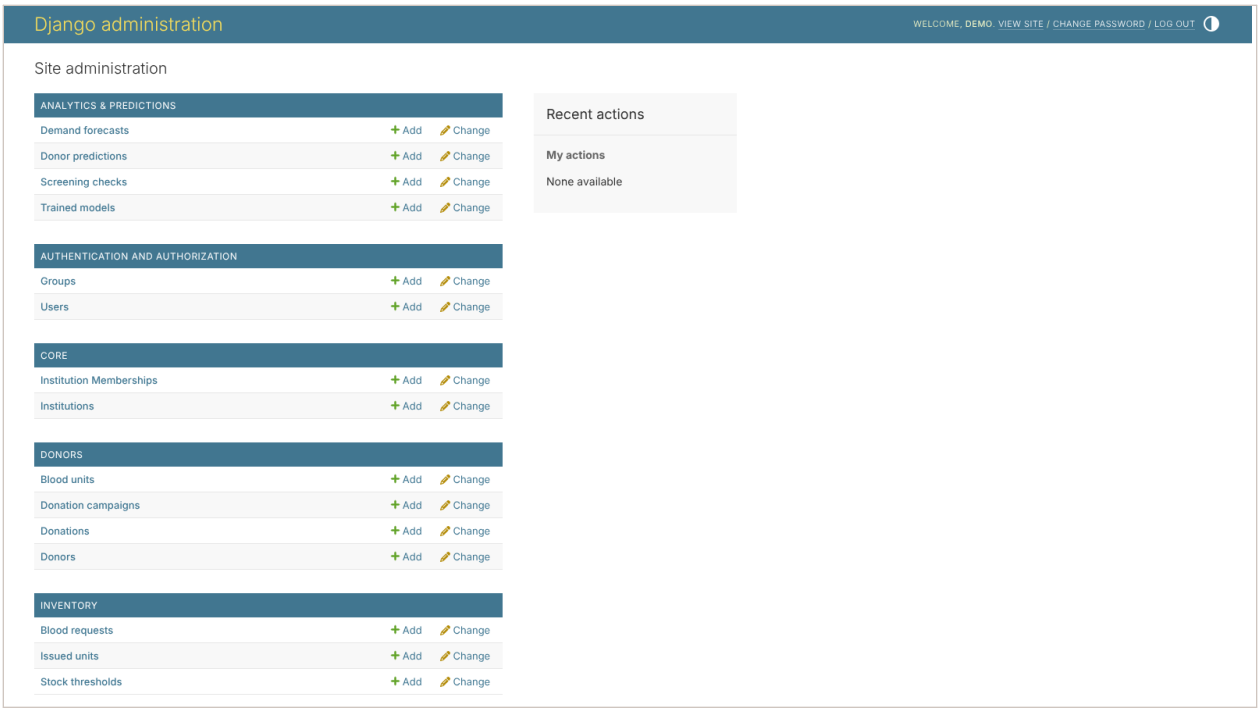


Figure 12. Django admin site listing all registered models, including Analytics & predictions.

5.2 Analytics & Predictions screens (new)

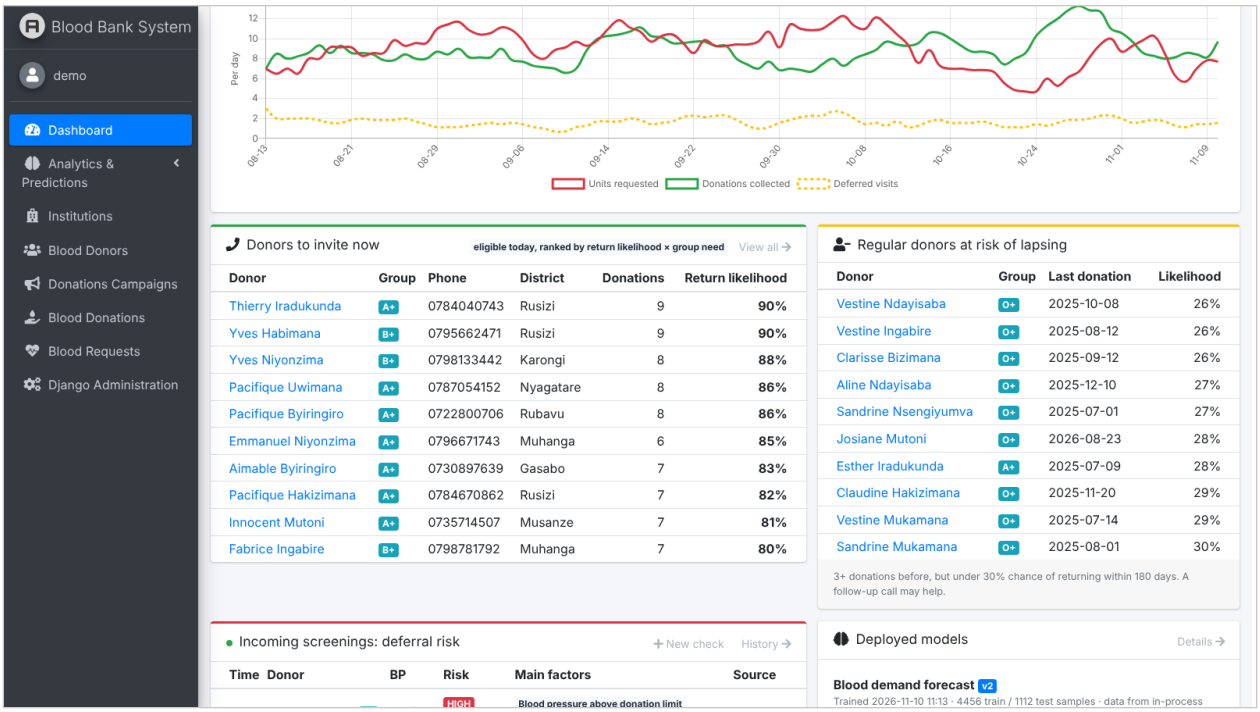


Figure 13. Dashboard prediction panels (lower half) — 90-day trends, donors to invite now and regular donors at risk of lapsing.

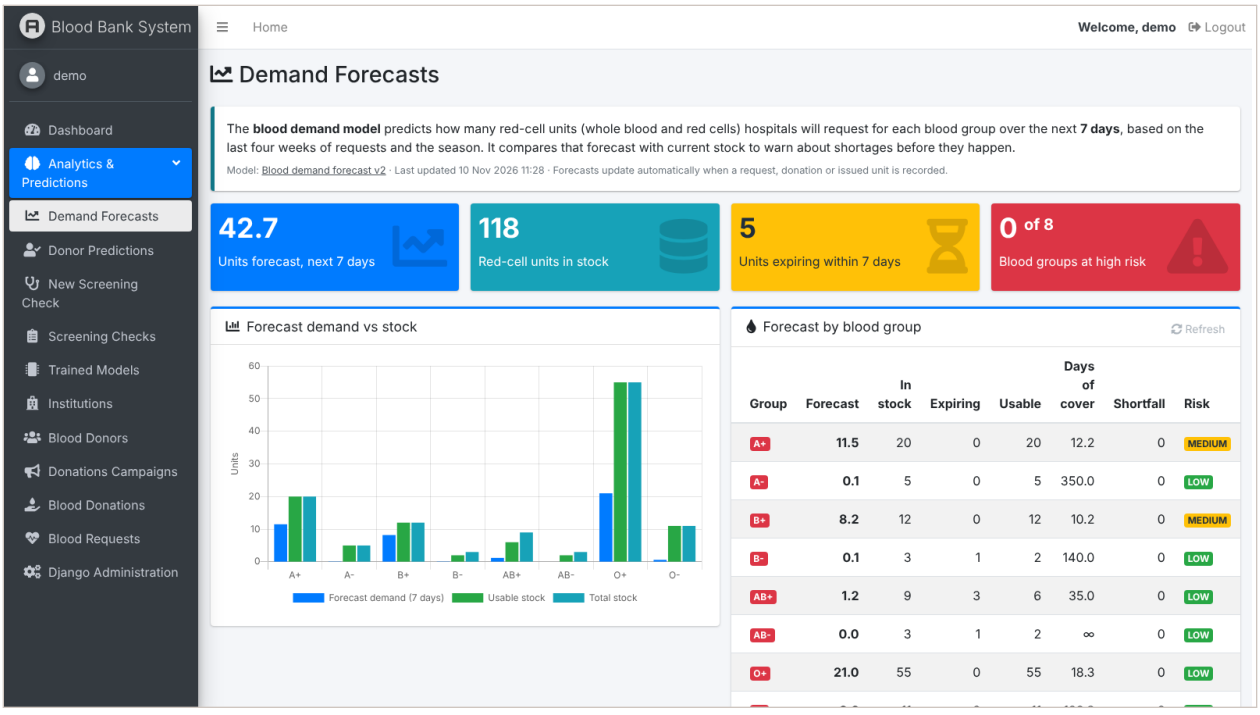


Figure 14. Analytics → Demand Forecasts: 7-day red-cell forecast, stock, days of cover and risk per blood group.

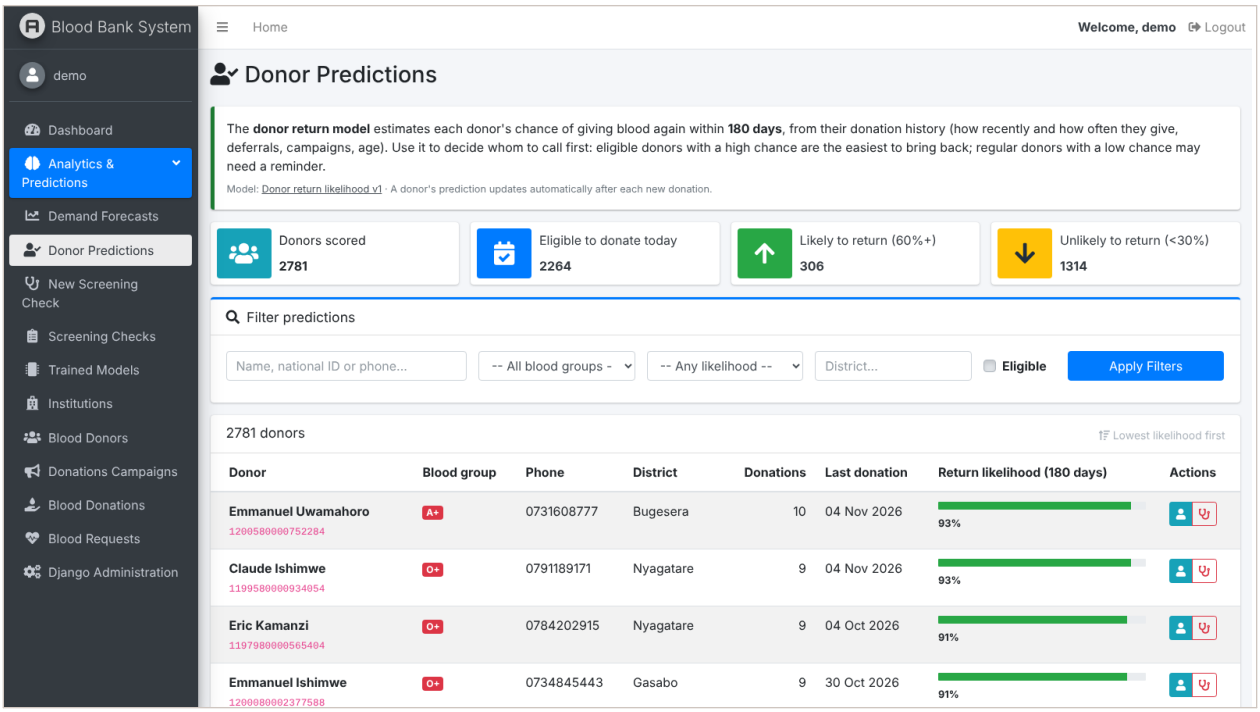


Figure 15. Analytics → Donor Predictions: return likelihood for every donor, with filters and quick actions.

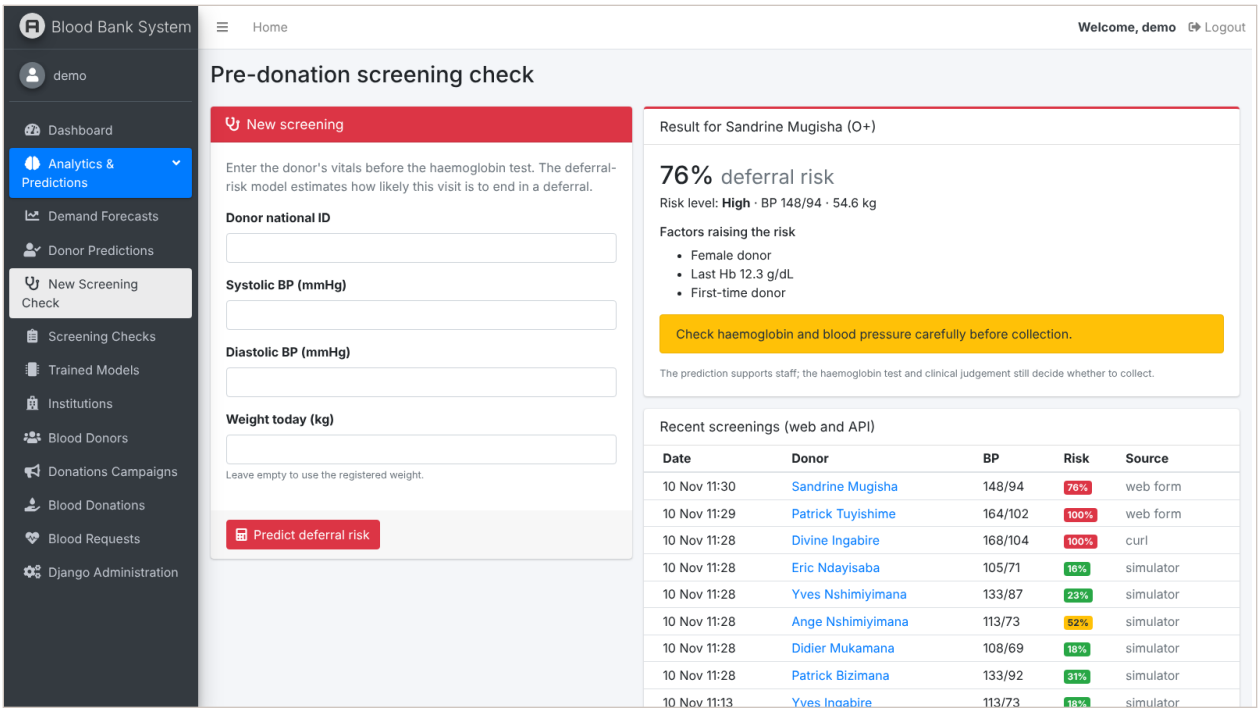


Figure 16. Analytics → New Screening Check: a deferral-risk prediction with the factors that raised it.

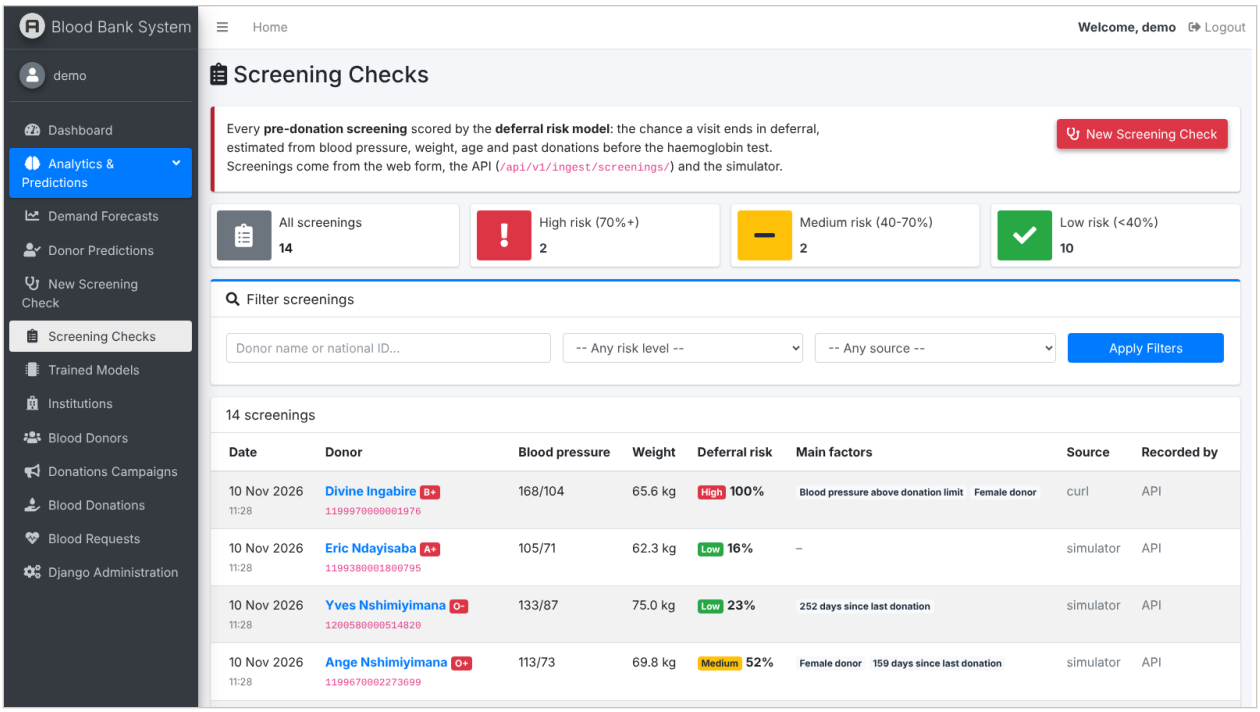


Figure 17. Analytics → Screening Checks: history of screenings from the web form, the API and the simulator.

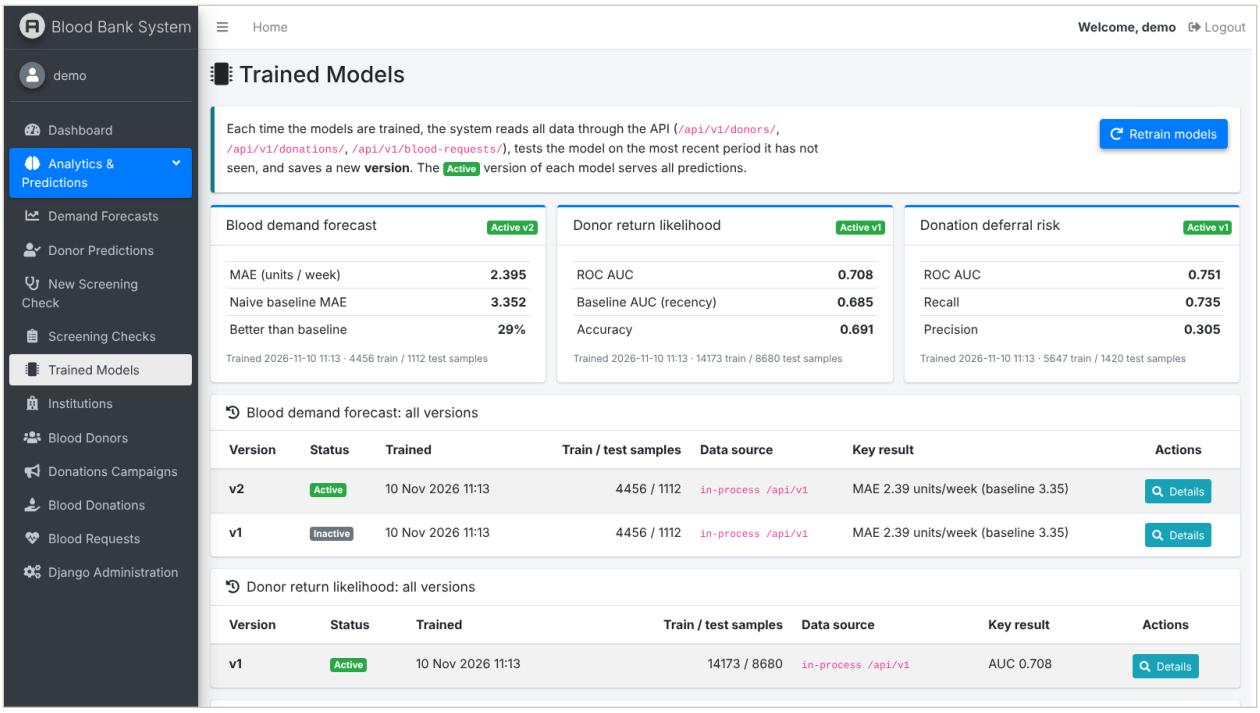


Figure 18. Analytics → Trained Models: active versions, test results, version history and the Retrain button.

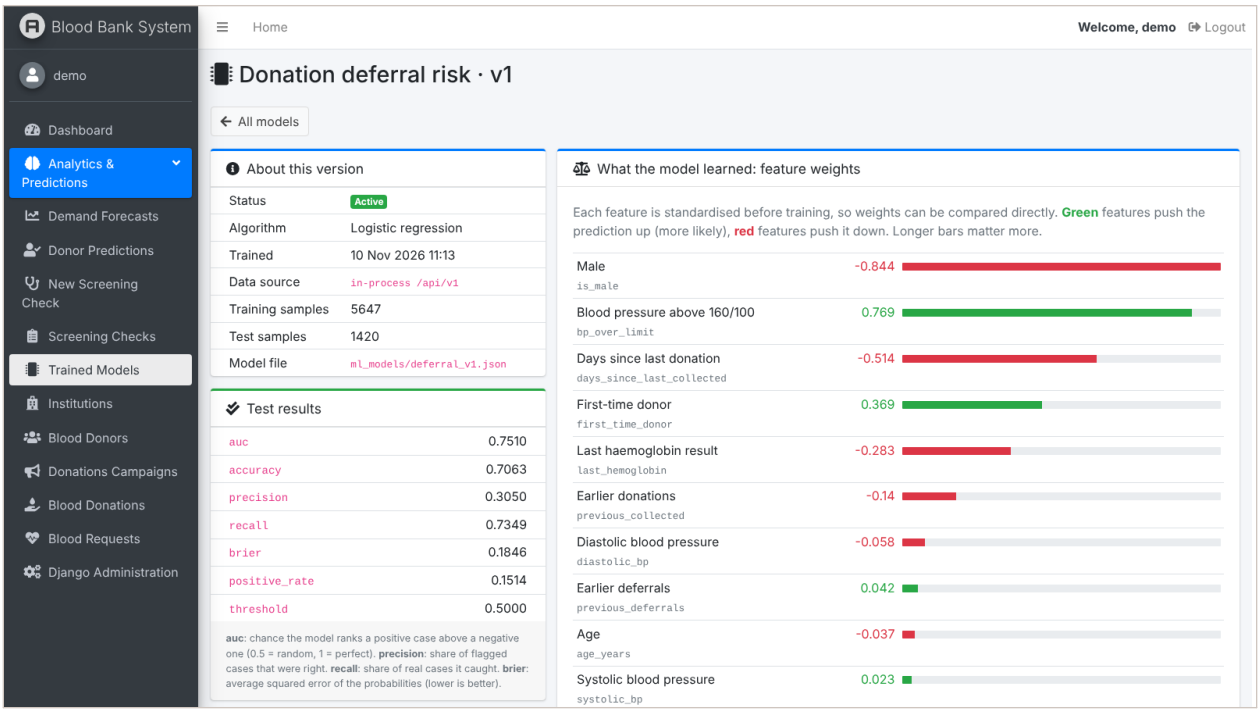


Figure 19. Trained model detail: test metrics and the feature weights the deferral model learned.

5.3 API screens (new)

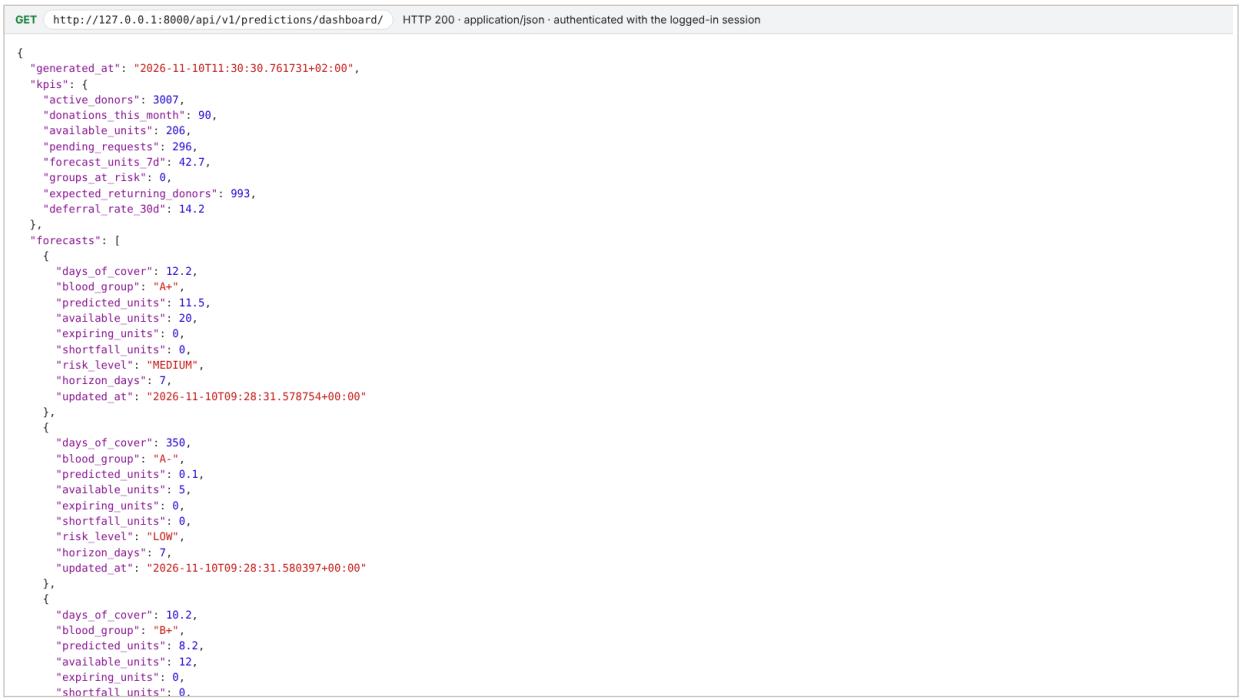


Figure 20. GET /api/v1/predictions/dashboard/ — the JSON behind the dashboard.

GET http://127.0.0.1:8000/api/v1/donors/?page_size=5 HTTP 200 · application/json · authenticated with the logged-in session


```
{
  "count": 3007,
  "page": 1,
  "next": "/api/v1/donors/?page_size=5&page=2",
  "results": [
    {
      "id": 1,
      "national_id": "1199180000000610",
      "first_name": "Patrick",
      "last_name": "Tuyishime",
      "gender": "M",
      "date_of_birth": "1991-03-21",
      "blood_group": "O+",
      "phone": "0798877065",
      "district": "Kicukiro",
      "weight_kg": 69.1,
      "is_active": true,
      "registered_at": "2025-10-22T07:00:00+00:00"
    },
    {
      "id": 2,
      "national_id": "1199970000001976",
      "first_name": "Divine",
      "last_name": "Ingabire",
      "gender": "F",
      "date_of_birth": "1999-12-18",
      "blood_group": "B+",
      "phone": "0725007072",
      "district": "Rusizi",
      "weight_kg": 65.6,
      "is_active": true,
      "registered_at": "2024-12-18T07:00:00+00:00"
    },
    {
      "id": 3,
      "national_id": "1200370000002899",
      "first_name": "Sandrine",
      "last_name": "Mugisha",
      "gender": "F",
      "date_of_birth": "2003-05-31",
      "blood_group": "O+",
      "phone": "0721437026",
```


Figure 21. GET /api/v1/donors/?page_size=5 — paginated donor records used for training.

GET http://127.0.0.1:8000/api/v1/blood-requests/?since=2026-11-01 HTTP 200 · application/json · authenticated with the logged-in session


```
{
  "count": 38,
  "page": 1,
  "next": null,
  "results": [
    {
      "id": 3071,
      "request_number": "RQ-2611-000BFF",
      "requestor_id": 9,
      "receiver_id": 1,
      "blood_group": "A+",
      "component": "WB",
      "quantity_units": 1,
      "urgency": "ROUTINE",
      "status": "FULFILLED",
      "request_date": "2026-11-01T19:16:00+02:00",
      "request_day": "2026-11-01",
      "required_by": "2026-11-05"
    },
    {
      "id": 3072,
      "request_number": "RQ-2611-000C02",
      "requestor_id": 6,
      "receiver_id": 1,
      "blood_group": "O+",
      "component": "WB",
      "quantity_units": 1,
      "urgency": "URGENT",
      "status": "FULFILLED",
      "request_date": "2026-11-02T07:55:00+02:00",
      "request_day": "2026-11-02",
      "required_by": "2026-11-03"
    },
    {
      "id": 3073,
      "request_number": "RQ-2611-000C03",
      "requestor_id": 5,
      "receiver_id": 1,
      "blood_group": "O+",
      "component": "WB",
      "quantity_units": 1,
      "urgency": "URGENT",
```


Figure 22. GET /api/v1/blood-requests/?since=2026-11-01 — date-filtered request records.

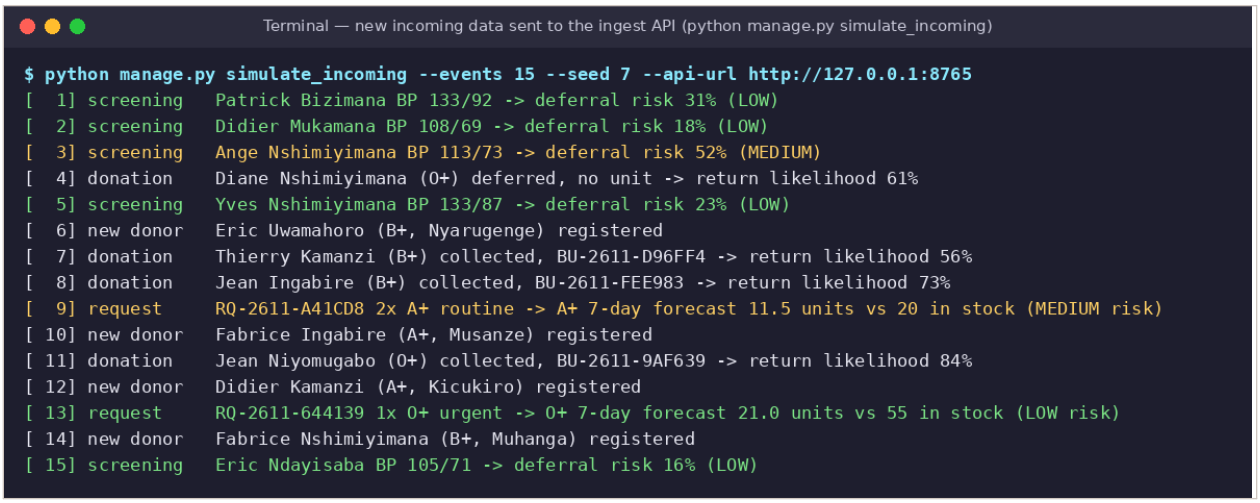


Figure 23. New incoming data sent over HTTP to the ingest API; each line shows the prediction returned.

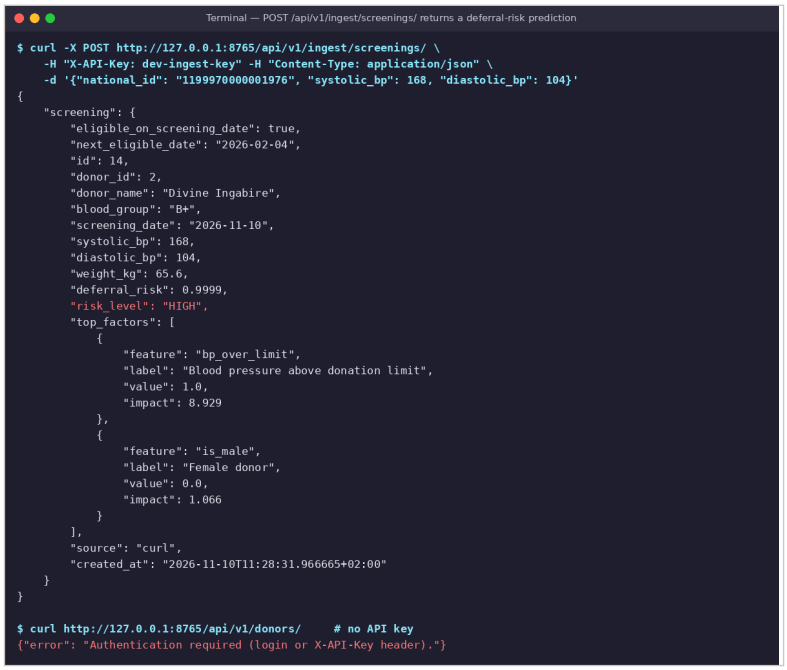


Figure 24. POST /api/v1/ingest/screenings/ returning a deferral risk with its factors, and HTTP 401 without an API key.

6. Testing

The project has **37 automated tests** (`python manage.py test`), all passing. They cover the business rules (donor age and weight, the 56-day interval, haemoglobin deferral, editing existing records), the views (login redirects, POST-only actions, issuing and un-issuing units), and the whole machine-learning path: the numpy algorithms on data with known answers, generating a synthetic history, training all three models through the API, checking that the demand model beats its baseline and both classifiers beat chance, API authentication and pagination, ingest endpoints returning predictions, signals refreshing predictions after a new donation, the analytics pages, and the staff-only retrain and roll-back actions.

7. Challenges and how they were resolved

The midterm challenges (coordinating schema changes across three people, Django defaults that did not match the project's URLs, and turning real-world rules into validation rather than field types) shaped how the final-exam work was approached. The new work brought its own lessons:

Synthetic data has to obey the system's own rules

Generated data that breaks validation is useless: it either fails to load or teaches the model impossible situations. The generator therefore builds each donor's history visit by visit, respecting age limits, the 56-day interval and campaign dates, and a sample of its output is re-validated with `full_clean()`. Running the existing `seed_demo` command the same way exposed a bug: after the 25th of each month it created donations dated after the current campaign had ended. It was fixed by capping those dates at the campaign's end.

Mixing blood components hid real shortages

The first demand forecast counted all components together. Plasma keeps for a year, so the plasma freezer made every blood group look well stocked even when red cells were short. Because plasma and platelets cannot replace red cells, the forecast and its stock comparison were restricted to red-cell products (whole blood and red cells), and the risk level was changed to *days of cover* after excluding units that expire within the forecast week.

A linear model cannot learn a hard clinical limit on its own

Deferral for blood pressure happens above a sharp threshold (160/100), but a linear model can only add a little risk per mmHg, so it under-rated clearly hypertensive donors. Adding an explicit "blood pressure above the limit" feature let the model learn the rule, raised the deferral model's AUC from about 0.73 to about 0.75, and made its explanations clearer.

Avoiding leakage and train/serve mismatch

Two subtle mistakes would have made the results look better than they are. Using the current visit's haemoglobin would give near-perfect deferral accuracy that is useless in practice, since it is only known after the test, so it was excluded. A random train/test split would let models learn from the future, so all evaluation uses the most recent period. Finally, training and live prediction share the same serializers and feature functions, so a deployed model never sees data shaped differently from what it was trained on.

Line endings and merge conflicts

When the final-exam branch was merged, one teammate's commit had converted almost every file to Windows line endings, so 1,853 files appeared changed and three files truly conflicted. The real conflicts were resolved by keeping the team's equivalent fixes, and a `.gitattributes` file now stores every text file with Unix line endings so the problem cannot recur.

8. Conclusion

The project meets all four final-exam requirements on top of the midterm foundations. Data is read through the project's own authenticated, paginated JSON API; three models are trained on a realistic synthetic history and evaluated honestly against baselines on a period they did not see; the models are deployed inside Django as versioned, traceable files with retrain and roll-back from the UI; and every new donation, request and screening is scored as it arrives, with the predictions shown on an upgraded dashboard and four dedicated analytics pages. The system now has four Django apps, fourteen entities with twenty-two documented relationships, 42 routes including 10 API endpoints, and 37 passing tests. The next step would be to retrain the same pipeline on a real blood bank's records.

9. Appendix: Running the system and demo data

Two management commands populate the database. The midterm `seed_demo` command creates a small, hand-crafted working set (4 institutions, 24 donors, 2 campaigns, 24 donations, 8 requests) that is ideal for demonstrating CRUD. It is too small to train models on, so the final exam adds `setup_demo`, which generates the two-year synthetic history, trains the three models through the API and sends a few incoming events:

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python manage.py migrate
python manage.py createsuperuser
python manage.py setup_demo          # synthetic data + training + incoming events
python manage.py runserver

# second terminal: live incoming data
python manage.py simulate_incoming --events 30 --interval 5 \
    --api-url http://127.0.0.1:8000
```

The README in the repository lists every command, the API endpoints and where each analytics feature is in the sidebar.

10. References

[1] Kundan Kumar. Final examination project requirements (course assignment brief). Web Technologies and API Services, AUCA, 2026.

[2] Kundan Kumar. Django Web Development — Midterm Examination: Project Work (course assignment brief). Web Technologies and API Services, AUCA, 2026.

[3] Django Software Foundation. Django Documentation. https://docs.djangoproject.com/en/stable/

[4] Django Software Foundation. Signals. https://docs.djangoproject.com/en/stable/topics/signals/

[5] ColorlibHQ. AdminLTE 3 Documentation. https://adminlte.io/docs/

[6] NumPy Developers. NumPy Documentation. https://numpy.org/doc/stable/

[7] Chart.js Contributors. Chart.js Documentation. https://www.chartjs.org/docs/

[8] T. Hastie, R. Tibshirani and J. Friedman. The Elements of Statistical Learning, 2nd ed. Springer, 2009 (ridge and logistic regression).

[9] T. Fawcett. An introduction to ROC analysis. Pattern Recognition Letters, 27(8):861–874, 2006.

[10] W3Schools. Django Tutorial. https://www.w3schools.com/django/